LEARNING TO PROGRAM · SCIENTIFIC ENGINEERING

Python for Engineering Intensive 3-Month Roadmap

From mathematical reasoning on paper to algorithms, reusable modules, engineering simulators, and a technical portfolio on GitHub.

Python | Mathematics | Numerical Methods | Aeronautics | Astrodynamics | GitHub Portfolio

> **Main objective:** by the end of the program, you should be able to open a blank page, take an engineering problem, define its inputs, states, equations, algorithm, validation, and outputs, and convert it into structured Python code.

Study plan prepared on August 18, 2026.

1. The central idea

The goal is not to learn Python as a collection of commands. The goal is to learn how to convert **mathematics and engineering into software**. That is why the course follows this chain:

PROGRAMMING → MATHEMATICS → NUMERICAL METHODS → ENGINEERING → AEROSPACE

And every problem will always be approached through the same process:

Problem on paper → Algorithm → Functions → Module → Validation → Plots → GitHub Project

Example: for a Hohmann transfer, we will not begin by using an astrodynamics library. First, we will implement the physics ourselves:

$$v_c = \sqrt{(\mu/r)}$$

$$a_t = (r_1 + r_2)/2$$

$$v_p = \sqrt{[\ \mu(2/r_1 - 1/a_t)\ ]}$$

```
hohmann_transfer.py

circular_velocity()
transfer_semimajor_axis()
transfer_velocity_perigee()
transfer_velocity_apogee()
delta_v_1()
delta_v_2()
transfer_time()
```

Then we will add visualization, tests, and finally compare the solution with a professional tool. This way, the underlying logic is learned rather than only how to use an API.

2. Overall structure of the 3 months

Suggested pace: **5-7 hours per week**, split into 45-60 minute sessions whenever possible.

Month	Objective	Outcome
Month 1	Programming logic and structured Python	Functions, modules, arrays, files, classes, and first tests.
Month 2	Computational mathematics and numerical methods	Algebra, derivatives, integrals, statistics, Monte Carlo, and ODEs.
Month 3	Application to aeronautics and space	Attitude, turbojet, orbital dynamics, transfers, and porkchop plots.

3. MONTH 1 - Learning to think like a programmer

Week	Programming	Mathematics	Project
1	Variables, if, for, while, functions	Algebra	Mathematical Calculator
2	Lists, tuples, dictionaries, arrays	Vectors and matrices	Vector Library
3	Functions, modules, imports	Trigonometry and coordinates	Coordinate Transform Toolkit
4	Classes, files, exceptions	Geometry / engineering	Mini Scientific Library

Week 1 - Logic, iteration, and functions

It is not enough to know what a `for` loop does; the goal is to learn **when a problem requires iteration**.

$$S = \Sigma_{i=1..N}\, i^2$$

```
total = 0
for i in range(1, N + 1):
    total += i**2
```

Then the algorithm is converted into a function, tested, and alternative approaches are discussed.

```
def sum_squares(n):
    ...

assert sum_squares(3) == 14
```

Complementary practice: Exercism and Project Euler.

Week 2 - Data structures, vectors, and matrices

A progressive introduction to NumPy, while initially avoiding letting libraries hide the algorithms. For example, before using `numpy.linalg.solve`, we will implement a simple Gaussian elimination method.

$$A\,x = b$$

```
import numpy as np
x = np.linalg.solve(A, b)
```

The objective is to understand what the library is doing and when to use it.

Week 3 - Functions, modules, and coordinate systems

```
coordinate_transforms/
├── rotations.py
├── spherical.py
├── frames.py
├── constants.py
└── tests/
```

We will implement the rotation matrices $R_x(\varphi)$, $R_y(\theta)$, $R_z(\psi)$, as well as functions such as:

```
def rot_x(phi):
    ...

def rot_y(theta):
    ...

def rot_z(psi):
    ...

def euler_to_rotation_matrix(phi, theta, psi):
    ...
```

> **Project:** Coordinate Transform Toolkit - Cartesian ↔ Spherical, NED ↔ Body, ECEF ↔ local frame, and Euler angles ↔ rotation matrix.

Week 4 - Classes and software organization

A class makes sense when there is a clear relationship between data, state, and behavior.

```
class Turbojet:
    def __init__(self, compressor_ratio, turbine_efficiency):
        self.compressor_ratio = compressor_ratio
        self.turbine_efficiency = turbine_efficiency

    def compressor_exit_temperature(self):
        ...

    def thrust(self):
        ...
```

We will also be able to represent physical objects such as an orbit or a vehicle.

4. MONTH 2 - Computational mathematics

Week	Mathematics	Programming	Project
5	Linear algebra and equations	NumPy / SciPy	Numerical Toolbox
6	Derivatives and integrals	SymPy / SciPy	Calculus Toolkit
7	Statistics	pandas / Monte Carlo	Uncertainty Simulator
8	ODEs	Euler / RK4 / solve_ivp	Dynamics Simulator

Week 5 - Numerical algebra

Own implementations before using libraries:

- Solving systems A x = b.
- Bisection.
- Newton-Raphson.
- Interpolation.
- Least squares.

$$x_{n+1} = x_n - f(x_n)/f'(x_n)$$

$$\hat{x} = (A^T A)^{-1} A^T b$$

Week 6 - Differential and integral calculus

$$f'(x) \approx [f(x+h) - f(x-h)] / (2h)$$

Numerical differentiation, rectangles, trapezoidal rule, and Simpson's rule. Then an introduction to `scipy.integrate` and SymPy for symbolic calculus.

Week 7 - Statistics and Monte Carlo

Mean, standard deviation, covariance, correlation, and distributions, applied to uncertainty simulations.

> **Mini project:** projectile range dispersion by varying velocity, angle, mass, drag, and wind across thousands of simulations.

```
for simulation in range(N):
    ...

# Outputs
mean_range
standard_deviation
percentile_95
```

Week 8 - Ordinary differential equations

$$dy/dt = f(t,y)$$

We will first implement Euler, then RK2, and finally RK4.

```
def rk4_step(f, t, y, dt):
    ...
```

Then we will compare the result with `scipy.integrate.solve_ivp`.

> **Project:** Falling Body Simulator with drag and outputs `altitude.dat`, `velocity.dat`, and `acceleration.dat`.

5. MONTH 3 - Aeronautics and space

Week	Topic	Project
9	Aircraft and attitude	Aircraft Attitude Simulator
10	Propulsion	Turbojet Performance Simulator
11	Astrodynamics	Orbital Mechanics Toolkit
12	Space mission	Launcher / Orbit Mission Simulator + Porkchop

Week 9 - Flight Mechanics

Euler angles φ, θ, ψ; transformation matrices; progressively adding states, velocities, forces, moments, and angular rates.

> **Aircraft-Attitude-Simulator:** Euler angles, rotation matrices, quaternions, body/NED transformations, IMU angular rates, and 3D visualization.

Week 10 - Turbojet

Atmosphere → Intake → Compressor → Combustion Chamber → Turbine → Nozzle

Calculation of thermodynamic stations and performance.

$$F = \dot{m}(V_e - V_0)$$

$$TSFC = \dot{m}_f / F$$

Parametric studies versus altitude, Mach, pressure ratio, TIT, and efficiencies.

Week 11 - Astrodynamics

Physical constants, the two-body problem, orbital elements, Cartesian ↔ Keplerian conversion, Hohmann transfers, inclination changes, escape velocity, sphere of influence, and Lambert's problem.

$$\ddot{r} = -\mu\, r / |r|^3$$

Propagation state:

$$x = [x, y, z, v_x, v_y, v_z]$$

Week 12 - Porkchop plot and interplanetary mission

Departure and arrival dates, propagation, Lambert's problem, C3, v∞, and ΔV will be combined.

```
for departure_date in departures:
    for arrival_date in arrivals:
        ...
        # Solve Lambert
        # Compute C3 / v_inf / delta-v
```

> **Outcome:** Earth → Mars Porkchop Plot as a portfolio project.

6. After the three months - PDEs and continuous-domain computation

The next stage will introduce partial differential equations using finite differences.

$$\partial T/\partial t = \alpha \, \partial^2 T/\partial x^2$$

$$T_i^{n+1} = T_i^n + \alpha(\Delta t/\Delta x^2)(T_{i+1}^n - 2T_i^n + T_{i-1}^n)$$

> **Initial project:** Heat Transfer Through an Aircraft Panel. Then: 2D heat equation, wave equation, and Poisson equation.

7. Recommended resources

MIT OpenCourseWare - Introduction to CS and Programming Using Python
Especially useful for computational thinking, branching, iteration, functions, and problem decomposition.
https://ocw.mit.edu/courses/6-100l-introduction-to-cs-and-programming-using-python-fall-2022/

Harvard CS50P - Introduction to Programming with Python
Very practical for conditionals, loops, exceptions, libraries, tests, file I/O, and object-oriented programming.
https://cs50.harvard.edu/python/

Exercism - Python Track
A collection of short exercises to build fluency, practice concepts, and get used to tests.
https://exercism.org/tracks/python

Project Euler
Mathematical reasoning and programming problems; especially useful once conditionals, loops, and functions are comfortable.
https://projecteuler.net/archives

Berkeley - Python Programming and Numerical Methods
A resource especially aligned with engineering: programming, numerical methods, algebra, ODEs, integration, interpolation, and more.
https://pythonnumericalmethods.berkeley.edu/

Official documentation
Python: https://docs.python.org/3/tutorial/
NumPy: https://numpy.org/doc/stable/user/absolute_beginners.html
SciPy: https://docs.scipy.org/doc/scipy/
SymPy: https://docs.sympy.org/latest/
pandas: https://pandas.pydata.org/docs/getting_started/
Matplotlib: https://matplotlib.org/stable/gallery/

Video and aerospace training
As a complement for numerical methods and astrodynamics, Alfonso González's material is useful for seeing implementations of orbital integration, Runge-Kutta, and orbital mechanics.
https://www.youtube.com/@AlfonsoGonzalezSpaceEngineering

Testing with pytest
We will introduce it when exercises begin to become reusable modules and we want to automatically validate results and edge cases.
https://docs.pytest.org/en/stable/getting-started.html

Astropy Coordinates
Professional reference for later comparison with our own coordinate conversions and representations.
https://docs.astropy.org/en/stable/coordinates/

8. Professional repositories and tools to study later

Project	Use	When to study it
RocketPy	Rocket trajectory and simulation	After our 1D/2D simulator
TudatPy	Astrodynamics, propagation, estimation, and mission design	After Two-Body and orbital elements
AeroSandbox	Aerodynamics, aircraft design, and optimization	Once NumPy/SciPy and code architecture are solid
RotorPy	6-DoF multirotor dynamics	After attitude and ODEs
JSBSim	Flight dynamics	After the attitude simulator
pyCycle	Thermodynamic engine modeling	After building our own turbojet model

Reference links: RocketPy - github.com/RocketPy-Team/RocketPy · TudatPy - docs.tudat.space · AeroSandbox - github.com/peterdsharpe/AeroSandbox · RotorPy - github.com/spencerfolk/rotorpy · JSBSim - jsbsim-team.github.io/jsbsim/python/ · pyCycle - github.com/OpenMDAO/pyCycle.

9. Target architecture for a GitHub project

```
orbital-mechanics-python/
├── README.md
├── requirements.txt
├── src/
│   ├── constants.py
│   ├── dynamics.py
│   ├── integrators.py
│   ├── orbital_elements.py
│   ├── maneuvers.py
│   └── plotting.py
├── tests/
│   ├── test_integrators.py
│   ├── test_orbital_elements.py
│   └── test_maneuvers.py
├── examples/
│   ├── circular_orbit.py
│   ├── eccentric_orbit.py
│   └── hohmann_transfer.py
├── notebooks/
│   └── earth_mars_analysis.ipynb
├── data/
└── figures/
```

The goal is to get used from the beginning to separating calculations, input/output, visualization, examples, and tests.

10. The four main portfolio repositories

Repository	Content
Python-Numerical-Methods	Algebra, root finding, derivatives, integrals, RK4, and Monte Carlo.
Aircraft-Dynamics-Python	Coordinates, attitude, atmosphere, and performance.
Turbojet-Performance-Simulator	Thermodynamic cycle, thrust, TSFC, and parametric studies.
Orbital-Mechanics-Python	Propagation, orbital elements, Hohmann, Lambert, and porkchop plots.

Optional additional project: **Rocket-Flight-Simulator**, incorporating variable mass, thrust, drag, gravity, and atmosphere, and evolving from 1D to 2D/3D.

11. How to go from an equation to a program

Before writing code, a minimal design sheet will be prepared.

```
PROBLEM
Calculate the trajectory of a projectile with drag.

INPUTS
mass, Cd, area, rho, v0, angle

STATES
x, z, vx, vz

EQUATIONS
dx/dt = vx
dz/dt = vz
dvx/dt = ...
dvz/dt = ...

ALGORITHM
1. Read parameters
2. Create initial state
3. Compute derivatives
4. Integrate state
5. Stop when z <= 0
6. Store results
7. Plot trajectory

OUTPUTS
time, position, velocity, maximum altitude, range
```

> **Fundamental habit:** do not start by programming. First translate the physical problem into inputs, states, equations, algorithm steps, and outputs.

12. Tutoring and review method

Exercises will be worked through using a review system similar to that of a technical team. When code is submitted, the following will be reviewed:

- Logic and mathematical correctness.
- Python style and naming.
- Edge cases.
- Efficiency and readability.
- Modularity.
- Testing.
- Which concept should be trained next.

The intention is to avoid immediately giving the full solution when the goal is for you to develop the logic independently.

13. Bootcamp Level 1 structure

1. Conditionals.
2. Loops.
3. Functions.

1. Lists, dictionaries, and arrays.
2. Scope.
3. Errors and exceptions.
4. NumPy.
5. Matplotlib.
6. Files.
7. Modules.
8. Classes.
9. Testing.

Each topic will follow the same pattern:

Theory → 2 simple exercises → 2 mathematical exercises → 1 engineering exercise → 1 mini project

Example for `for`:

1+2+...+N → factorial → Taylor series → iterative trajectory → Monte Carlo

14. What it means to complete the program successfully

The goal is not to be able to say “I know Python.” The goal is to be able to look at an equation, a spreadsheet, or an engineering problem and confidently begin to structure it computationally.

> **Target outcome:** tools such as RocketPy, Tudat, Kalman filters, engines, structures, or any scientific library should stop looking like black boxes and instead become software built from blocks that you already know how to design and validate.

This document is the reference map. Practical classes, exercises, partial solutions, code reviews, and projects will be developed progressively to avoid passive learning.